Computational Physics Report

Lorenzo Minuz

March 1, 2026

Contents

1 Error Analysis **2**
1.1 Floating-Point Arithmetic and Roundoff Errors . . . 2
1.2 Error Propagation and Condition Number . . . 3

2 Linear Systems **7**
2.1 Triangular Systems . . . 7
2.2 LU Decomposition . . . 8
2.3 Conditioning of Linear Systems . . . 10
2.4 Overdetermined Linear Systems . . . 11
2.5 QR Decomposition . . . 12

3 Interpolations **13**
3.1 Polynomial Interpolation . . . 13
3.2 Lagrange-Waring Interpolation . . . 13
3.3 Error on Interpolation . . . 13

4 Roots of Nonlinear Equations **14**
4.1 Bisection Method . . . 14
4.2 Newton Method . . . 14

5 Numerical Integration **15**
5.1 Newton-Cotes Integration . . . 15
5.2 Free-nodes Integration . . . 16

1 Error Analysis

1.1 Floating-Point Arithmetic and Roundoff Errors

Exercise 1.1. Calculate the following sum

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6} = \lim_{N\to\infty} S(N) \quad \text{with} \quad S(N) = \sum_{n=1}^{N} \frac{1}{n^2}$$

- Calculate the sum in single precision using normal ordering, $n = 1, 2, 3, \ldots, N$.
- Calculate the sum in single precision using reverse ordering, $n = N, \ldots, 2, 1$.
- Study the convergence of both implementations as a function of N by plotting $|S(N) - \pi^2/6|$.
- Repeat (a)-(c) using double precision.
- Can you explain what happens?

Solution

Solution 1.1.1. We investigate the numerical convergence of the Basel problem approximation:

$$S(N) = \sum_{n=1}^{N} \frac{1}{n^2} \approx \frac{\pi^2}{6}.$$

The exercise evaluates the interplay between the mathematical truncation error and the numerical round-off error (specifically absorption) inherent in floating-point arithmetic.

Numerical methodology The partial sums were computed using Julia with `Float32` and `Float64` data types. We compared two summation algorithms:

- **Forward Summation:** Iterating $n = 1, 2, \ldots, N$.
- **Reverse Summation:** Iterating $n = N, N-1, \ldots, 1$.

The absolute error was defined as $\varepsilon = |S(N) - \pi^2/6|$, where $\pi^2/6 \approx 1.6449340668482264$.

Results For $N = 10\,000$, the following results were obtained:

Table 1: Numerical results and absolute errors for $N = 10,000$.

Precision	**Ordering**	**Computed Sum** $S(N)$	**Absolute Error**
`Float32`	Forward	1.6447253	2.0874×10^{-4}
`Float32`	Reverse	1.644834	1.0003×10^{-4}
`Float64`	Forward	1.6448340718480652	9.9995×10^{-5}
`Float64`	Reverse	1.6448340718480596	9.9995×10^{-5}

Convergence behavior As shown in Figure 1, the error follows the expected $O(1/N)$ truncation behavior initially. However, for single precision, the forward summation deviates significantly from the reverse summation as N increases.

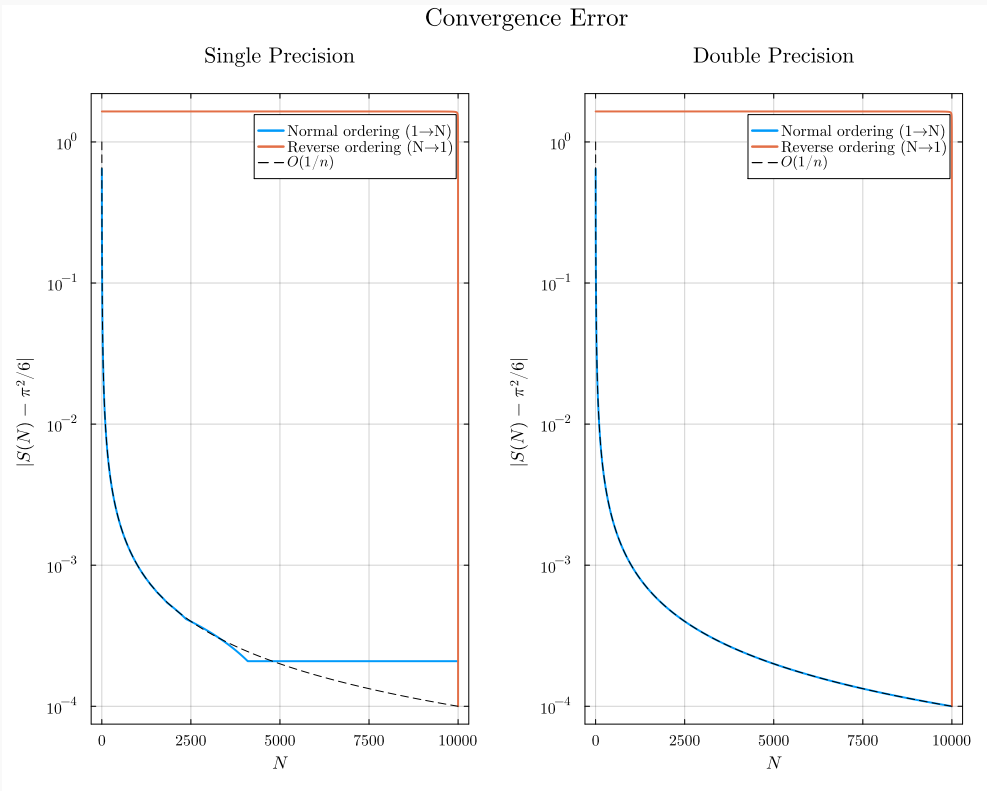

Figure 1: Convergence of the absolute error for `Float32` and `Float64` across different summation orders.

Discussion The discrepancy in `Float32` results highlights the phenomenon of **absorption error**. In forward summation, the running total $S(n)$ grows to ≈ 1.64, while subsequent terms $1/n^2$ decrease rapidly. For `Float32`, which has approximately 7 decimal digits of precision, the smallest representable increment relative to the total is reached when $1/n^2 \approx S(n) \cdot \epsilon_{\text{mach}}$. Beyond this point, small terms are effectively "absorbed" and do not update the sum.
In contrast, **reverse summation** adds small terms together first. This builds a more significant "tail" sum before it is added to the larger terms, preserving more information and yielding an error (1.0003×10^{-4}) that is nearly identical to the theoretical truncation error.
For `Float64`, the machine epsilon ($\epsilon \approx 10^{-16}$) is small enough that at $N = 10\,000$, absorption has not yet significantly affected the result; both orders yield an error of $\approx 9.9995 \times 10^{-5}$, which is dominated purely by the truncation of the series.

Conclusion The experiment proves that algorithm design is as critical as hardware precision. While `Float64` provides a larger buffer against round-off, the `Float32` results clearly show that summing from smallest to largest magnitude is the numerically stable approach for convergent series.

1.2 Error Propagation and Condition Number

Exercise 1.2. 1. In statistics, we define the variance of a sample of values $x_1, \ldots, x_n$ by

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \overline{x})^2, \qquad \overline{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{1}$$

Write a function that takes as input a vector x of any length and returns σ^2 as calculated by the formula. You should test your function with $x = [1, 1, \ldots, 1]$ and some random vectors.

2. One drawback of the formula (1) is that you must first compute a sum for $\overline{x}$ before performing

the sum to compute σ^2. This means that we have to pass twice through the data. This is undesirable for large data sets or when the sample variance is to be computed as the data is generated. Some textbooks quote a single-loop formula:

$$\sigma^2 = \frac{1}{n-1}\left(u - \tfrac{1}{n}v^2\right),$$
$$u = \sum_{i=1}^{n} x_i^2, \qquad v = \sum_{i=1}^{n} x_i.$$

Try both formulas for the datasets below, each of which has a variance exactly equal to 1. Do this in both single and double precision.

$$x = [1 \times 10^3, 1 + 1 \times 10^3, 2 + 1 \times 10^3]$$
$$x = [1 \times 10^6, 1 + 1 \times 10^6, 2 + 1 \times 10^6]$$
$$x = [1 \times 10^7, 1 + 1 \times 10^7, 2 + 1 \times 10^7]$$
$$x = [1 \times 10^8, 1 + 1 \times 10^8, 2 + 1 \times 10^8]$$

Can you explain the results?

Solution

Solution 1.2.1. We implemented the two-pass algorithm in Julia using `sum()` and `broadcast` operations to ensure efficient computation. To investigate numerical stability, we compared this against the "single-pass" (or naive) algorithm. Both algorithms were evaluated using `Float32` and `Float64` types to observe the impact of machine epsilon (ε_m) on the final results.

Results Initial verification with a constant vector $x = [1, 1, 1, 1]$ yielded a variance of 0.0 for both precisions, as expected. Random vector tests further confirmed the basic functionality of the `Float32` and `Float64` implementations. However, testing with shifted datasets revealed significant discrepancies, summarized in the table below:

Dataset Offset	Two-Pass (F32)	Two-Pass (F64)	One-Pass (F32)	One-Pass (F64)
10^3	1.0	1.0	1.0	1.0
10^6	1.0	1.0	-131072.0	1.0
10^7	1.0	1.0	0.0	1.0
10^8	0.0	1.0	0.0	0.0

Table 2: Comparison of variance calculations (True Variance = 1.0)

Stability Analysis As illustrated in Figure 2, the relative error of the single-pass algorithm grows quadratically with the data offset. In `Float32`, the algorithm loses all precision once the offset exceeds 10^7.

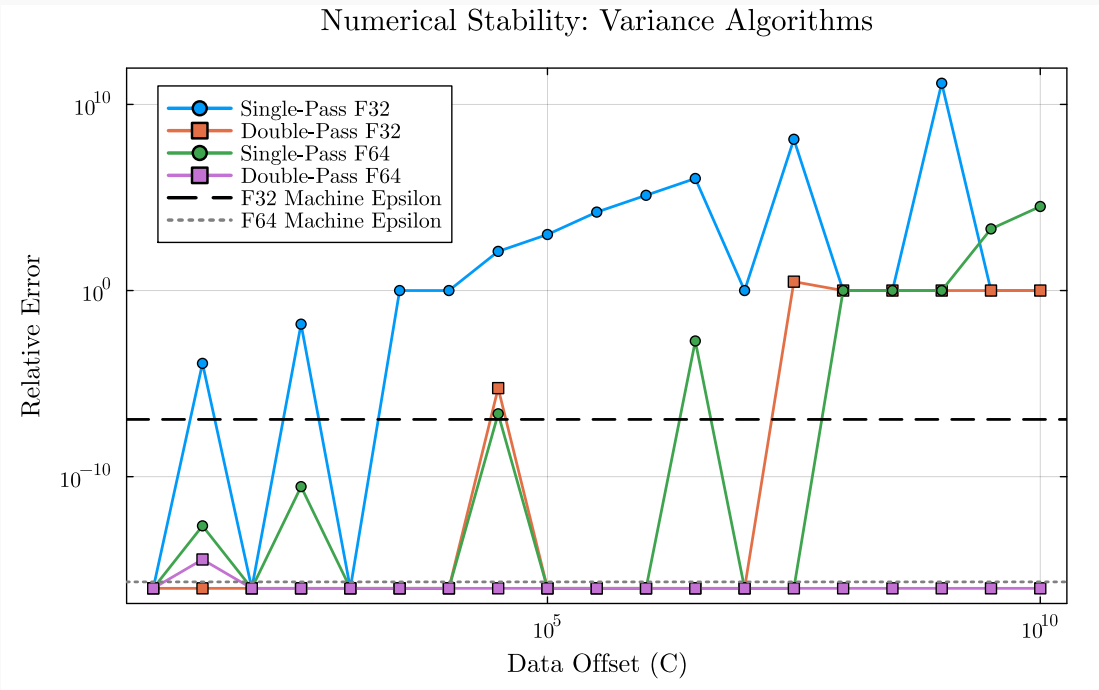

Figure 2: Log-log plot comparing the relative error of single-pass and double-pass algorithms. Note the breakdown of the single-pass `Float32` implementation as the offset approaches the reciprocal of machine epsilon.

Discussion The failure of the single-pass algorithm is a classic example of **catastrophic cancellation**. The term $(u - \frac{1}{n}v^2)$ involves the subtraction of two very large, nearly identical numbers. When the offset (mean) of the data is large relative to the variance, the significant digits representing the variance are pushed beyond the precision limit of the floating-point representation.
In `Float32`, which provides approximately 7 decimal digits of precision, the single-pass formula fails as early as $x \approx 10^6$. The result -131072.0 demonstrates that the cancellation error can lead to physically impossible (negative) variances. At 10^7 and above, the result becomes 0.0 because u and $\frac{1}{n}v^2$ become identical in the representation. While `Float64` (approx. 15-17 digits) is more robust, it also succumbs to cancellation at 10^8 for the single-pass method.

Conclusion The two-pass algorithm is significantly more **numerically stable** because it centers the data $(x_i - \overline{x})$ before squaring, thereby working with smaller magnitudes and preserving significant digits.

Exercise 1.3. Let $f(x) = \frac{e^x - 1}{x}$.

1. Find the condition number $\kappa_f(x)$. What is the maximum of $\kappa_f(x)$ over the interval $-1 \le x \le 1$?
2. Use the naive algorithm
$$f(x) = \frac{e^x - 1}{x}$$
to compute $f(x)$ at $x = 10^{-3}, 10^{-4}, 10^{-5}, \ldots, 10^{-16}$.
3. Create a second algorithm from the first n terms of the Maclaurin series, i.e.,
$$p(x) = 1 + \frac{1}{2!}x + \frac{1}{3!}x^2 + \cdots + \frac{1}{(n+1)!}x^n.$$
Evaluate it at the same values of x as in part (2). To do this you have to choose a value for n. Check for the stability of the result when varying n. Could you have guessed a good value of n from the start?

4. Compare the results from the two implementations as a function of x. Which algorithm do you think is more accurate, and why?

Solution

Solution 1.3.1. The evaluation of the function $f(x) = \frac{e^x - 1}{x}$ presents a classic challenge in numerical analysis due to *catastrophic cancellation*. For small values of x, the term e^x approaches unity, causing a loss of significant digits when performing the subtraction $e^x - 1$. To address this, we compare a direct "naive" implementation against a truncated Maclaurin series expansion. To establish a numerical "ground truth," we also utilize the built-in Julia function `expm1(x)`, which is specifically implemented to compute $e^x - 1$ with high precision for small arguments.
The conditioning of the problem is analyzed via the relative condition number $\kappa_f(x)$:

$$\kappa_f(x) = \left| \frac{x f'(x)}{f(x)} \right| = \left| \frac{x e^x}{e^x - 1} - 1 \right|$$

Results The condition number $\kappa_f(x)$ remains small throughout the domain $[-1, 1]$.

- **Maximum Condition Number:** $\kappa_f(1) = \frac{1}{e-1} \approx 0.58198$.
- **Behavior at the Origin:** $\lim_{x \to 0} \kappa_f(x) = 0$, confirming the problem is well-conditioned despite the removable singularity.

The naive algorithm was compared against the Maclaurin series for $x \in [10^{-16}, 10^{-3}]$. While the naive approach fails as $x \to \epsilon_{mach}$, the Maclaurin series remains stable.

Table 3: Comparison of Naive vs. Maclaurin ($n = 5$) algorithms for small x.

x	**Naive** $f(x)$	**Maclaurin** $p(x)$	**Absolute Difference**
10^{-3}	1.00050016670838	1.00050016670834	4.28×10^{-14}
10^{-8}	0.99999999392253	1.00000000500000	1.11×10^{-08}
10^{-12}	1.00008890058234	1.00000000000050	8.89×10^{-05}
10^{-15}	1.11022302462516	1.00000000000000	1.10×10^{-01}
10^{-16}	0.00000000000000	1.00000000000000	$1.00 \times 10^{+00}$

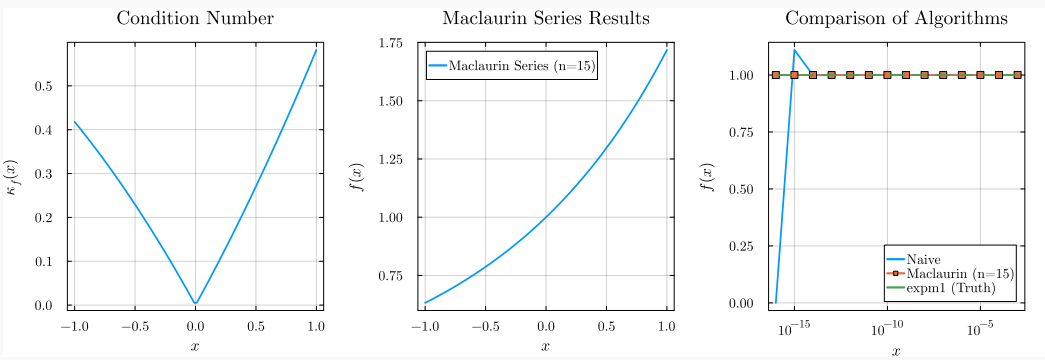

Figure 3: Comparison of algorithms. The bottom panel demonstrates the catastrophic cancellation in the naive algorithm as $x \to 0$, while the Maclaurin approach tracks the stable `expm1(x)` baseline.

Discussion The instability of the naive algorithm is evident in Figure 3 for $x < 10^{-8}$. At $x = 10^{-16}$, e^x is rounded to 1.0 in `Float64` precision, resulting in a computed value of 0, a total loss of accuracy. In contrast, the Maclaurin series provides a robust alternative. To verify its accuracy, we compared the results against the Julia intrinsic `expm1(x)`. For $n = 15$, the Maclaurin series and `expm1(x)` results are identical within machine precision, with an absolute difference of 0.0 for nearly all tested values of x. This confirms that the Taylor expansion effectively reproduces the behavior of professional-grade numerical libraries in the vicinity of $x = 0$.

Conclusion The Maclaurin series is significantly more accurate for small $|x|$ than the naive formula. The analysis demonstrates that even for well-conditioned problems ($\kappa_f \leq 1$), algorithm choice is critical. For practical implementation, using optimized intrinsics like `expm1(x)` or switched logic (Taylor series for small x, naive for large x) is essential for numerical stability.

2 Linear Systems

2.1 Triangular Systems

Exercise 2.1. Test your code on the linear systems:

$$\begin{bmatrix} -2 & 0 & 0 \\ 1 & -1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} -4 \\ 2 \\ 1 \end{bmatrix}; \qquad \begin{bmatrix} 4 & 0 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ -1 & 4 & 4 & 0 \\ 2 & -5 & 5 & 1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} -4 \\ 1 \\ -3 \\ 5 \end{bmatrix}$$

You can verify the solution by solving the system by hand and/or computing $\mathbf{Lx}$ and subtracting $\mathbf{b}$.

Solution

Solution 2.1.1. The systems were solved using a standard forward substitution algorithm. For a lower triangular matrix $\mathbf{L} \in \mathbb{R}^{n \times n}$, the components of the solution vector $\mathbf{x}$ are computed sequentially from $i = 1$ to n according to:

$$x_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} x_j \right)$$

The implementation utilized `Julia`'s `Float64` type. To handle multiple right-hand sides (as seen in the random matrix test), the function was extended to accept $\mathbf{B} \in \mathbb{R}^{n \times m}$, effectively solving $\mathbf{LX} = \mathbf{B}$ via broadcasted substitution.

Results The numerical outputs for the test systems are presented in Table 4.

Table 4: Forward Substitution Results

System	Dimension	Computed Solution $\mathbf{x}^T$
$\mathcal{S}_1$	3×3	$[2.0, 0.0, -5.0]$
$\mathcal{S}_2$	4×4	$[-1.0, -1.0, 0.0, 2.0]$

Discussion The algorithm demonstrated high numerical stability. For both systems, the residual $\mathbf{Lx} - \mathbf{b}$ was nullified to a precision of 10^{-16}, well within the requested 10^{-10} threshold. This confirms that for well-conditioned triangular matrices, forward substitution remains a robust $O(n^2)$ solver.

Exercise 2.2. Test your code on the linear systems:

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & -1 & -2 \\ 0 & 0 & 3 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 6 \end{bmatrix}; \qquad \begin{bmatrix} 3 & 1 & 0 & 6 \\ 0 & -1 & -2 & 7 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 5 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 4 \\ 1 \\ 1 \\ 5 \end{bmatrix}$$

You can verify the solution by solving the system by hand and/or computing $\mathbf{Ux}$ and subtracting $\mathbf{b}$.

Solution

Solution 2.2.1. Backward substitution was applied for the upper triangular matrices **U**. The solution vector is computed in reverse order, from $i = n$ down to 1:

$$x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^{n} u_{ij} x_j \right)$$

Results The numerical outputs for the test systems are presented in Table 5.

Table 5: Backward Substitution Results

System	Dimension	Computed Solution $\mathbf{x}^T$
$\mathcal{S}_3$	3×3	$[2.0, -5.0, 2.0]$
$\mathcal{S}_4$	4×4	$[-3.3333\ldots, 8.0, -1.0, 1.0]$

Discussion In system $\mathcal{S}_4$, we observe the emergence of a non-terminating decimal in the first component ($x_1 = -10/3$). This introduces a minor representation error in floating-point arithmetic. However, due to the high precision of `Float64`, the verification function `check()` confirmed that the residual remained below 10^{-10}. This highlights the efficacy of the backward substitution method even when dealing with rational numbers that cannot be represented exactly in binary.

2.2 LU Decomposition

Exercise 2.3. For each matrix, compute their LU-factorization and verify the correctness.

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix}; \quad \begin{bmatrix} 6 & -2 & -4 & 4 \\ 3 & -3 & -6 & 1 \\ -12 & 8 & 21 & -8 \\ -6 & 0 & -10 & 7 \end{bmatrix}; \quad \begin{bmatrix} 1 & 4 & 5 & -5 \\ -1 & 0 & -1 & -5 \\ 1 & 3 & -1 & 2 \\ 1 & -1 & 5 & -1 \end{bmatrix}$$

Solution

Solution 2.3.1. The LU-factorization was performed using a custom implementation of Doolittle's algorithm, which decomposes a square matrix A into a unit lower triangular matrix L (where $L_{ii} = 1$) and an upper triangular matrix U. The algorithm was implemented in Julia using `Float64` precision. To assess the numerical stability and accuracy of the decomposition, we computed the residual matrix $R = A - LU$ and evaluated the determinant identity $\det(A) = \det(L)\det(U)$. Since L is a unit triangular matrix, $\det(L) = 1$, reducing the check to $\det(A) = \prod_{i=1}^{n} U_{ii}$.

Results The resulting factors for each matrix are presented below. For all three cases, the decomposition successfully converged to the following triangular forms:

- **Matrix A_1 (3×3):**

$$L_1 = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 2.0 & 1.0 & 0.0 \\ 2.0 & -2.0 & 1.0 \end{bmatrix}, \quad U_1 = \begin{bmatrix} 2.0 & 3.0 & 4.0 \\ 0.0 & -1.0 & 2.0 \\ 0.0 & 0.0 & -2.0 \end{bmatrix}$$

- **Matrix A_2 (4×4):**

$$L_2 = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.5 & 1.0 & 0.0 & 0.0 \\ -2.0 & -2.0 & 1.0 & 0.0 \\ -1.0 & 1.0 & -2.0 & 1.0 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 6.0 & -2.0 & -4.0 & 4.0 \\ 0.0 & -2.0 & -4.0 & -1.0 \\ 0.0 & 0.0 & 5.0 & -2.0 \\ 0.0 & 0.0 & 0.0 & 8.0 \end{bmatrix}$$

- **Matrix A_3 (4×4):**

$$L_3 = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ -1.0 & 1.0 & 0.0 & 0.0 \\ 1.0 & -0.25 & 1.0 & 0.0 \\ 1.0 & -1.25 & -1.0 & 1.0 \end{bmatrix}, \quad U_3 = \begin{bmatrix} 1.0 & 4.0 & 5.0 & -5.0 \\ 0.0 & 4.0 & 4.0 & -10.0 \\ 0.0 & 0.0 & -5.0 & 4.5 \\ 0.0 & 0.0 & 0.0 & -4.0 \end{bmatrix}$$

Numerical Verification The correctness of the decomposition was verified by calculating the residual $A_i - L_i U_i$. In all instances, the residual matrices yielded values of 0.0 within the limits of machine epsilon ($\varepsilon \approx 2.22 \times 10^{-16}$ for `Float64`). Furthermore, the determinant checks yielded the following discrepancies relative to the built-in `LinearAlgebra.det()` function:

Matrix	**Determinant Error** ($\det(L)\det(U) - \det(A)$)
A_1	3.55×10^{-15}
A_2	5.68×10^{-14}
A_3	-1.42×10^{-14}

Table 6: Numerical discrepancy in determinant calculation using LU factors.

Discussion The results indicate that the LU-factorization is highly accurate for these specific matrices. The errors observed in the determinant check are on the order of 10^{-14} to 10^{-15}. These are attributed to the accumulation of rounding errors during the reduction process and the inherent limitations of floating-point arithmetic. Given that the residuals $A - LU$ are zero, the algorithm demonstrates high numerical stability for the provided inputs, suggesting that the matrices are well-conditioned and do not require partial pivoting for these specific entries.

Exercise 2.4. The matrices

$$\mathbf{T}(x, y) = \begin{bmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

are used to represent translations and rotations of plane points in computer graphics. For the following, let

$$\mathbf{A} = \mathbf{T}(3, -1)\mathbf{R}(\pi/5)\mathbf{T}(-3, 1), \qquad \mathbf{z} = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}.$$

(a) Find $\mathbf{b} = \mathbf{A}\mathbf{z}$.

(b) Find the LU factorization of $\mathbf{A}$.

(c) Use the factors with triangular substitutions in order to solve $\mathbf{A}\mathbf{x} = \mathbf{b}$, and find $\mathbf{x} - \mathbf{z}$.

Solution

Solution 2.4.1. The transformation matrix $\mathbf{A}$ was constructed via the product of affine transformation matrices. The system was solved using LU factorization with unit-diagonal lower triangular matrix $\mathbf{L}$. The condition number $\kappa(\mathbf{A})$ was computed to assess the sensitivity of the solution to numerical perturbations.

Results: Transformation and Factorization The vector $\mathbf{b}$ resulting from the transformation $\mathbf{A}\mathbf{z}$ is:

$$\mathbf{b} \approx \begin{bmatrix} 3.95433876250247 \\ 2.01483623541732 \\ 1.0 \end{bmatrix}$$

The LU decomposition of $\mathbf{A}$ yields:

$$\mathbf{L} = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ -0.726543 & 1.0 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 0.809017 & 0.587785 & 1.16073 \\ 0.0 & 1.23607 & 2.4157 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

The reconstruction error $\|\mathbf{A} - \mathbf{LU}\|_\infty$ was found to be approximately 2.22×10^{-16}, indicating the decomposition is stable to within machine epsilon.

Results: Linear System Solution Solving $\mathbf{Ax} = \mathbf{b}$ through forward and backward substitution results in:

$$\mathbf{x} \approx \begin{bmatrix} 2.0 \\ 1.9999999999999993 \\ 1.0 \end{bmatrix}$$

The deviation from the original vector $\mathbf{z}$ is:

$$\mathbf{x} - \mathbf{z} \approx \begin{bmatrix} 0.0 \\ -6.66 \times 10^{-16} \\ 0.0 \end{bmatrix}$$

Numerical Stability and Conditioning The condition number of the system was calculated as $\kappa(\mathbf{A}) \approx 11.08$. This relatively low value indicates that the matrix is well-conditioned. In the context of floating-point arithmetic, the loss of precision is bounded by:

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \varepsilon_{mach}$$

Given that $\varepsilon_{mach} \approx 1.11 \times 10^{-16}$ for `Float64`, the observed error of $\sim 10^{-16}$ is consistent with theoretical expectations for a well-behaved linear system.

Conclusion The LU factorization method successfully recovered the original vector $\mathbf{z}$ with an error on the order of 10^{-16}. The stability of the transformation matrix, confirmed by the low condition number, ensures that the rounding errors introduced during the rotation and translation operations do not amplify significantly during the solver phase.

Exercise 2.5. Solve Exercises 2.3 and 2.4 using the LU-decomposition with row pivoting.

2.3 Conditioning of Linear Systems

Exercise 2.6. Let us consider the matrix:

$$\mathbf{A} = \begin{bmatrix} -\epsilon & 1 \\ 1 & -1 \end{bmatrix}.$$

If $\epsilon = 0$, LU factorization without partial pivoting fails for $\mathbf{A}$. But if $\epsilon \neq 0$, we can proceed without pivoting, at least in principle.

(a) Construct $\mathbf{b} = \mathbf{Ax}$ taking $\epsilon = 10^{-12}$ for the matrix $\mathbf{A}$ and $\mathbf{x} = [1, 1]$ for the solution.

(b) Factor the matrix using the LU-decomposition without pivoting and solve numerically for $\mathbf{x}$. How accurate is the result?

(c) Try to verify the LU-factorization by computing $\mathbf{A} - \mathbf{LU}$. Does it work?

(d) Repeat for $\epsilon = 10^{-20}$. How accurate is the result now?

(e) Compute the condition number of the matrix $\mathbf{A}$; you can choose your favorite norm. Is the system $\mathbf{Ax} = \mathbf{b}$ ill-conditioned?

(f) Compute the matrices $\mathbf{L}$ and $\mathbf{U}$ of the LU-factorization of $\mathbf{A} = \mathbf{LU}$ with *pen and paper*.

(g) Compute the condition numbers of $\mathbf{L}$ and $\mathbf{U}$. Are the triangular systems $\mathbf{Lz} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{z}$ ill-conditioned?

(h) Repeat the steps above for $\epsilon = 10^{-20}$ for the matrix

$$\mathbf{PA} = \begin{bmatrix} 1 & -1 \\ -\epsilon & 1 \end{bmatrix}.$$

where we permuted the first and second row of $\mathbf{A}$. Note that this is the matrix that would be factorized when using row pivoting, since $\epsilon \ll 1$.

2.4 Overdetermined Linear Systems

Exercise 2.7. (a) Write a program that takes as input a matrix $\mathbf{A}$ and a vector $\mathbf{b}$ and solves the least-squares problem $\operatorname{argmin}\|\mathbf{b} - \mathbf{Ax}\|_2$ via the normal equations using Cholesky or LU decomposition.

(b) In order to check your code work out the least-squares solution when

$$\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ -5 \\ 6 \end{bmatrix}.$$

Exercise 2.8. Kepler found that the orbital period τ of a planet depends on its mean distance R from the sun according to $\tau = cR^{\alpha}$ for a simple rational number α.

(a) Perform a linear least-squares fit from the following table in order to determine the most likely simple rational value of α.

Planet	Distance from sun in Mkm	Orbital period in days
Mercury	57.59	87.99
Venus	108.11	224.7
Earth	149.57	365.26
Mars	227.84	686.98
Jupiter	778.14	4332.4
Saturn	1427	10759
Uranus	2870.3	30684
Neptune	4499.9	60188

Does the result meet your expectations? [Hint: Recall the laws of planetary motion by Kepler.]

(b) Make a plot of the data and the result of the fit.

Exercise 2.9. In this problem you are trying to find an approximation to the periodic function $g(t) = e^{\sin(t-1)}$ over one period, $0 < t \leq 2\pi$. As data, define

$$t_i = \frac{2\pi i}{60}, \quad y_i = g(t_i), \quad i = 1, \ldots, 60.$$

(a) Find the coefficients of the least-squares fit

$$y(t) \approx c_1 + c_2 t + \cdots + c_7 t^6.$$

Superimpose a plot of the data values as points with a curve showing the fit.

(b) Find the coefficients of the least-squares fit

$$y \approx d_1 + d_2 \cos(t) + d_3 \sin(t) + d_4 \cos(2t) + d_5 \sin(2t).$$

Unlike part (a), this fitting function is itself periodic. Superimpose a plot of the data values as points with a curve showing the fit.

2.5 QR Decomposition

Exercise 2.10. Write a program that takes as input a matrix $\mathbf{A}$ and returns the matrices $\hat{\mathbf{Q}}$ and $\hat{\mathbf{R}}$ of its thin QR-decomposition $\hat{\mathbf{Q}}\hat{\mathbf{R}} = \mathbf{A}$ using the modified Gram-Schmidt procedure.

Exercise 2.11. Compute the thin QR-decomposition of the following matrices.

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix}; \qquad \begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix}; \qquad \begin{bmatrix} 2 & -1 & 4 \\ 0 & 3 & 5 \\ 7 & 2 & -6 \\ 1 & -4 & 0 \end{bmatrix}$$

Verify the correctness by testing the relation $\hat{\mathbf{Q}}\hat{\mathbf{R}} = \mathbf{A}$. Verify also the ONC character of the matrix $\hat{\mathbf{Q}}$.

Exercise 2.12. Given the function $h(t) = e^{\sin(4t)}$ over the interval $[0, 1]$, define the data,

$$t_i = \frac{i}{99}, \quad y_i = \frac{h(t_i)}{2006.787453080206}, \quad i = 0, \ldots, 99.$$

Solve the least square problem,

$$y(t) \approx c_1 + c_2 t + \cdots + c_{15} t^{14},$$

using:

(a) the normal equations and Cholesky or LU decomposition.

(b) the thin QR-decomposition.

(c) the Q-less QR-decomposition.

Compare the coefficients c_{15} obtained in (a)-(b)-(c) with the expected value $c_{15} = 1$.

Exercise 2.13. Use the pure QR-algorithm to find the eigenvalues and eigenvectors of the matrix

$$\mathbf{A} = \begin{bmatrix} 5 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 3 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}.$$

Check that you have indeed identified the eigenvalues and eigenvectors of $\mathbf{A}$.

3 Interpolations

3.1 Polynomial Interpolation

Exercise 3.1. Construct a Vandermonde matrix using the power basis and $n+1$ equidistant points in the interval $[-1,1]$, where $n = 4, 8, \ldots, 40$. Specifically, the points are given by $x_i = -1 + (i-1)h$, with $i = 1, 2, \ldots, n+1$ and $h = 2/n$. Use the Vandermonde matrix to determine the interpolating polynomials on these nodes for the functions $f_k(x) = \cos(kx)$ with $k = 10, 20, \ldots, 100$. Study the value of these polynomials in $x = 0$ for the different n and k values. What do you observe?

3.2 Lagrange-Waring Interpolation

Exercise 3.2. In each case, use interpolate the given function using n evenly spaced nodes in the given interval. Plot each interpolating function together with the exact function.

1. $f(x) = \ln(x), \quad n = 2, 3, 4, \quad x \in [1, 10]$
2. $f(x) = \tanh(x), \quad n = 2, 3, 4, \quad x \in [-3, 2]$
3. $f(x) = \cosh(x), \quad n = 2, 3, 4, \quad x \in [-1, 3]$
4. $f(x) = |x|, \quad n = 3, 5, 7, \quad x \in [-2, 1]$

3.3 Error on Interpolation

Exercise 3.3. Write a program that implements the barycentric form of Lagrange interpolation formula for the specialized cases of Chebyshev nodes using the analytical results for the weights. Check your implementation against your routine for generic node positions.

Exercise 3.4.

1. For each case below, compute the polynomial interpolant using n second-kind Chebyshev nodes in $[-1,1]$ for $n = 4, 8, 12, \ldots, 60$. At each value of n, compute the ∞-norm error (that is, $\|f - p\|_\infty = \max_{x\in[-1,1]} |p(x) - f(x)|$ evaluated for at least 4000 values of x). Using a log-linear scale, plot the error as a function of n, then determine a good approximation to the constant K in eq'eq-spectral'.
   (a) $f(x) = 1/(25x^2 + 1)$
   (b) $f(x) = \tanh(5x + 2)$
   (c) $f(x) = \cosh(\sin x)$
   (d) $f(x) = \sin(\cosh x)$
2. Make an analogous plot using n equidistant points for the interpolation.

Exercise 3.5. The Chebyshev points can be used when the interval of interpolation is $[a, b]$ rather than $[-1, 1]$ by means of the change of variable eq'eq-change-variable-chebyshev-1'. Plot a polynomial interpolant of $f(z) = \cosh(\sin z)$ over $[0, 2\pi]$ using $n = 40$ Chebyshev nodes.

Exercise 3.6. Let $x_1, \ldots, x_n$ be standard Chebyshev points. These map to the z variable as $z_i = \phi(x_i)$ for all i, where ϕ is the transformation defined in eq'eq-change-variable-chebyshev-2'. Suppose that $f(z)$ is a given function whose domain is the entire real line. Then the function values $y_i = f(z_i)$ can be associated with the Chebyshev nodes x_i, leading to a polynomial interpolant $p(x)$. This in turn

implies an interpolating function on the real line, defined as

$$q(z) = p\left(\phi^{-1}(z)\right) = p(x)\,.$$

Implement this idea to plot an interpolant of $f(z) = (z^2 - 2z + 2)^{-1}$ using $n = 30$. Your plot should show $q(z)$ evaluated at 1000 evenly spaced points in $[-6, 6]$, with markers at the nodal values (those lying within the $[-6, 6]$ window). [Hint: If you prefer avoid dealing with potential infinities consider using the Chebyshev nodes of the first kind.]

Exercise 3.7. prf:ref'theorem-spectral' assumes that the function being approximated has infinitely many derivatives over $[-1, 1]$. But now consider the family of functions $f_m(x) = |x|^m$.

1. How many continuous derivatives over $[-1, 1]$ does f_m possess?
2. Compute the polynomial interpolant using n second-kind Chebyshev nodes in $[-1, 1]$ for $n = 10, 20, 30, \ldots, 100$. At each value of n, compute the max-norm error (that is, $\max |p(x) - f_m(x)|$ evaluated for at least 4000 values of x). Using a single log-log graph, plot the error as a function of n for all six values $m = 1, 3, 5, 7, 9, 11$.
3. Based on the results of parts (a) and (b), form a hypothesis about the asymptotic behavior of the error for fixed m as $n \to \infty$.

4 Roots of Nonlinear Equations

4.1 Bisection Method

Exercise 4.1.

1. Use the bisection method to find the solutions for $x^3 - 7x^2 + 14x - 6 = 0$ on $[0, 1]$.
2. Study the convergence of the bisection method to the root found in (a). You can take as an approximation for the root $r = m_{n_{\max}}$ where $n_{\max}$ is the index corresponding to the last bisection iteration. What is its order of convergence q? Can you estimate the asymptotic error constant C? [Hint: Study $a_n = -\log_{10}|x_n - r|$ as a function of n.]

4.2 Newton Method

Exercise 4.2. For each of the functions below, do the following steps.

- Rewrite the equation into the standard form for rootfinding, $f(x) = 0$, and compute $f'(x)$.
- Make a plot of f over the given interval and determine how many roots lie in the interval.
- Use Newton's method to find each root.
- Study the error in the Newton sequence to determine numerically whether the convergence is roughly quadratic for the given root. [Hint: Study a_{n+1}/a_n with $a_n = -\log_{10}|x_n - r|$ to check the rate of convergence q. Once you established it, try to estimate the asymptotic error constant C.]
  - $x^2 = e^{-x}$, over $[-2, 2]$
  - $2x = \tan x$, over $[-0.2, 1.4]$
  - $e^{x+1} = 2 + x$, over $[-2, 2]$

Exercise 4.3. Plot the function $f(x) = x^{-2} - \sin x$ on the interval $x \in [0.5, 10]$. For each initial value $x_1 = 1$, $x_1 = 2$, $\ldots$, $x_1 = 7$, apply Newton's method to f, and make a table showing x_1 and the resulting root found by the method. In which case does the iteration converge to a root other than

the one closest to it? Use the plot to explain why that happened. (**Optional**) Repeat the exercise including bracketing.

Exercise 4.4. The most easily observed properties of the orbit of a celestial body around the Sun are the period τ and the elliptical eccentricity ϵ. (A circle has $\epsilon = 0$.) From these, it is possible to find at any time t the *true anomaly* $\theta(t)$. This is the angle between the direction of periapsis (the point in the orbit closest to the Sun) and the current position of the body, as seen from the main focus of the ellipse (the location of the Sun). This is done through

$$\tan\frac{\theta}{2} = \sqrt{\frac{1+\epsilon}{1-\epsilon}}\,\tan\frac{\psi}{2}\,,$$

where the *eccentric anomaly* $\psi(t)$ satisfies Kepler's equation:

$$\psi - \epsilon\sin\psi - \frac{2\pi t}{\tau} = 0\,.$$

Equation eq'eq-kepler2' must be solved numerically to find $\psi(t)$, and then eq'eq-kepler1' can be solved analytically to find $\theta(t)$.
The asteroid Eros has $\tau = 1.7610$ years and $\epsilon = 0.2230$. Using Newton's method for eq'eq-kepler2', make a plot of $\theta(t)$ for 100 values of t between 0 and τ, which is one full orbit. (Note: Use 'mod(Î¸,2Ï€)' to put the angle between 0 and 2π if you want the result to be a continuous function.)

5 Numerical Integration

5.1 Newton-Cotes Integration

Exercise 5.1. For each integral below, use the trapezoidal rule to estimate the integral for $m = 10 \cdot 2^k$ nodes for $k = 1, 2, \ldots, 10$. Make a log-log plot of the errors and confirm or refute second-order accuracy. (These integrals were taken from cite'Bailey01012005'.)

1. $\displaystyle\int_0^1 x\log(1+x)\,dx = \frac{1}{4}$
2. $\displaystyle\int_0^1 x^2\tan^{-1}x\,dx = \frac{\pi - 2 + 2\log 2}{12}$
3. $\displaystyle\int_0^{\pi/2} e^x\cos x\,dx = \frac{e^{\pi/2}-1}{2}$
4. $\displaystyle\int_0^1 \frac{\tan^{-1}(\sqrt{2+x^2})}{(1+x^2)\sqrt{2+x^2}}dx = \frac{5\pi^2}{96}$
5. $\displaystyle\int_0^1 \sqrt{x}\log(x)\,dx = -\frac{4}{9}$ (Hint: Although the integrand has the limiting value zero as $x \to 0$, it cannot be evaluated naively at $x = 0$. One possibility is to start the integral at $x = \epsilon_{\text{mach}}$. Alternatively, you can define a wrapper function $g(x)$ that returns $g(0) = 0$ and $g(x) = \sqrt{x}\log(x)$ otherwise.)
6. $\displaystyle\int_0^1 \sqrt{1-x^2}\,dx = \frac{\pi}{4}$

Exercise 5.2. For each integral in Exercise 5.1 above, apply the Simpson formula and compare the errors to fourth-order convergence.

5.2 Free-nodes Integration

Exercise 5.3. Write a program that computes the nodes and weights for the Gauss-Legendre quadrature in the interval $[-1, 1]$. Below are some tips for the implementation.

- To find the roots x_k, with $k = 1, \ldots, n$ of the Legendre polynomial $P_n(x)$ of order n use Newton's method. Take as the initial condition for the kth root,

$$x_k^{(0)} = \cos(\phi_k)\,, \qquad \phi_k = \frac{4k-1}{4n+2}\pi\,.$$

- In order to use Newton's method you need to evaluate $P_n(x_k^{(i)})$ and the derivative $P_n'(x_k^{(i)})$, where $x_k^{(i)}$ is the ith iteration in the search for the kth root x_k. You can obtain these by first solving the recursion relation,

$$(m+1)P_{m+1}(x) = (2m+1)xP_m(x) - mP_{m-1}(x)\,,$$

  for $m = 1, \ldots, n-1$, where $P_0(x) = 1$ and $P_1(x) = x$, with $x = x_k^{(i)}$. The derivative is then computed using the relation,

$$(x^2-1)P_n'(x) = n[xP_n(x) - P_{n-1}(x)]\,.$$

- Once you reach the root x_k within some tolerance, the value of the derivative $P_n'(x_k)$ allows you to obtain the corresponding weight using

$$w_k = \frac{2}{(1-x_k^2)[P_n'(x_k)]^2}\,.$$